Reproducing “Near-Duplicate Detection for Web Crawling”

Name Surname

2026-08-01

Abstract This paper aims to reproduce the findings of the “Near-Duplicate Detection for Web Crawling” work, using PySpark to scale the proposed algorithm, under different datasets, feature models and parameters.

Introduction

Detecting duplicates in a dataset is crucial for tasks such as web crawling or AI model training, as it frees up computational resources. A straightforward approach to solve this involves hashing the data and comparing all rows via a Cartesian product. However, this approach is fragile: minor syntax differences in the data might produce entirely different hashes, which is undesirable. Moreover, the time complexity of a Cartesian product is $O(n^2)$, making it unfeasable for large datasets. The approach proposed by Manku et al. [1] suggest to identify near-duplicates instead. This method aims to generate similar hashes for similar data, leveraging this property to optimize the comparison algorithm. It’s also tested whether the algorithm is robust enough to find semantical near-duplicates, using a transformer-based embedding model. Lastly, some of the cited feature models and the proposed fingerprint matching algorithm are benchmarked against a variety of datasets, showcasing the efficiency of the suggested algorithm.

Methodology

The methodology proposed by Manku et al. [1] can be divided into a two-part pipeline.

1. Fingerprint Generation

The objective of this first phase is to transform text into a single, fixed-length binary value, referred to as a *fingerprint*. Specifically, similar data must corrispond to similar fingerprints (i.e. having a low Hamming distance). To achieve this, Charikar’s SimHash [2] is adopted: each text is converted into a set of fixed-length binary values called *features*, which are then assembled to form a fingerprint. A set of features is reduced into a fingerprint by selecting the most frequent bit value at each position: 1 if the majority of bits are 1 at that index, 0 otherwise.

Selecting the appropriate feature model and fingerprint size for a given dataset is important. The original paper proposes various feature models, such as *Shingles* and *TF-IDF*, ultimately selecting the latter with a fingerprint size of 64 bits. In this work, four feature models are tested and compared: *Bag of Words*, *Shingles*, *TF-IDF* and *Embeddings*, using fingerprint sizes ranging from 32 bits up to 512 bits.

2. Fingerprint Matching

Now every text in the dataset has a fingerprint that represents it. The goal for this phase is to compare fingerprints from two datasets, called *corpus* and *query* for simplicity sake, in order to find pairs of fingerprints which Hamming distance is less than K, a fixed integer value. The reference paper [1] approaches this problem by splitting each fingerprint into at least $K + n$ *partitions*, for $n > 0$. This grants a useful property: if two fingerprints are near-duplicate by a factor of K, then at least n of their $K + n$ partitions must be equal (the opposite doesn't hold - other partitions might have more than K different bits). A *schema* describes how partitions should be combined to efficiently find near-duplicate fingerprints. The optimal schema is derived by the following formula:

$$X(f,k,d) = \begin{cases} 1 & \text{if } \ d < \tau \\ \min_{r>k}\binom{r}{k} \cdot X\left(\frac{fk}{r}, k, d - \frac{(r-k)f}{r}\right) & \text{otherwise} \end{cases}$$

where f is the size of each fingerprint, K is the maximum allowed Hamming distance, d is the log_2 size of the dataset and $p_m in$ is the maximum allowed candidate set size (in log_2).

Once the partitions are composed, Manku et al. [1] approach can be summerized as follows:
1. Each partition is sorted and compressed, using an Huffman code on the prefix of the fingerprints
2. Partitions are distributed to workers, so that partitions with matching schemas are sent to the same worker
3. Perform a binary search on all *query* rows, probing *corpus* - matching rows are marked as candidates
4. The hamming distance of each candidate row is calculated, filtering rows whose distance is over *K*.

Implementation

This implementation uses the PySpark [3] framework, which allows to trivially scale the computation to clusters, for large datasets. What follows are the most interesting parts.

Feature Models

The implemented feature models are *Bag Of Words*, *Shingles*, *TF-IDF* and *Embeddings*. All of them are based on PySpark's DataFrames, using native methods from the package `pyspark.sql.functions`. When necessary, Pandas UDF are adopted.

Bag of Words

For a given text, its features are the individual words it contains. Hence, two texts are considered near-duplicate if they contain about the same words. This approach is computationally efficient and simple to implement, but it fails to preserve spatial locality. For example, the sentence "The bank is after the office" yields the exact same features as "The office is after the bank".

Shingles

Generally, a shingle is a contiguous sequence of n items from a data source; in this context, it refers to an overlapping contiguous sequence of n characters. Shingles are generated using a sliding window algorithm with a window size of n. This model preserves locality to some extent, but it generates a large number of features, which can negatively impact performance. To reduce the feature count, a non-overlapping variant is considered. Additionally, word-based shingles (as opposed to traditional character-based ones) are evaluated.

TF-IDF

This is a refined version of *Bag of Words* that accounts for both the frequency of a word within a specific text (DF) and its rarity across the entire dataset (IDF):

$$\text{TF-IDF}(term, document, Dataset) = TF(term, document) * IDF(term, Dataset)$$

TF stands for Term Frequency, which assigns higher importance to words that appear frequently in a given text:

$$TF(term, document) = \frac{\text{How many times term appears in document}}{\text{How many words document has}}$$

IDF stands for Inverse Document Frequency, which assigns higher weight to words that appear in fewer distinct documents. This filters out very common words, such as conjunctions and verbs:

$$IDF(term, Dataset) = \log(\frac{\text{Total number of documents in Dataset}}{\text{Number of documents that contain term}})$$

This is the feature model chosen in the original paper [1]. In PySpark, this model is implemented using `pyspark.ml.features.IDF` and `pyspark.ml.features.CountVectorizer`. Despite its higher accuracy, it's computationally more expensive than previous models, which were "local" in the sense that computing the features of a single text did not require information about the entire dataset. Furthermore, maintenance introduces additional overhead, as the weights of each term must be recomputed whenever the dataset changes.

Embeddings

An embedding model is a machine learning algorithm that maps text into a vector capturing its semantic meaning. In this study, a transformer-based model, *Qwen3-embedding-0.6b* [4], is used and executed locally using Ollama. Each weight of the generated embeddings is quantized to 0/1, taking the median of all weights as pivot, becoming a bit of the result fingerprint. Although suboptimal, this is required in order to build binary fingerprints. Processing embeddings of long texts can require large amount of memory for the KV cache. While all previous models rely on syntax to extract features, embedding models operate on semantics, allowing for the detection of semantical near-duplicate text.

FastSimHash

Charikar's SimHash algorithm [2] takes as input a list of hashes and returns a fingerprint, obtained by taking the most popular bit value for each index of the input hashes. This can be done in plain Python, or using matrix column sums with Numpy/Pandas. This can be hard to compute, if the feature model generates many hashes or the input text is large. In order to speed this process, the FastSimHash algorithm has been developed [5]. ... FastSimHash cannot be used for TF-IDF, because TF-IDF works with floating point weights.

Fingerprint Detection with RDDs

The detection algorithm from the original paper [1] has been replicated in PySpark using RDDs. In particular, *query* and *corpus* fingerprints are partitioned following the schema provided by the paper formula. All partitions are distributed across workers, based on their index: *query* and *corpus* partitions with matching index are sent to the same worker, using `repartition(...)` and `cogroup(...)` After sorting *corpus* partitions, for each *query* partition value, *corpus* partitions are probed using `bisect`, a Python module for binary searches. Sorting and performing binary searches requires to load all *corpus* partitions in memory. Matching partition values are returned to the driver as candidates, whose

Hamming distance will be computed and compared with K. The Huffman code compression is not implemented: in this work K is not fixed, and it's expensive to compute all the Huffman codes of the *corpus* partition for each iteration. However, it is certainly recommended for production environments where fingerprints are stored, with a fixed K value.

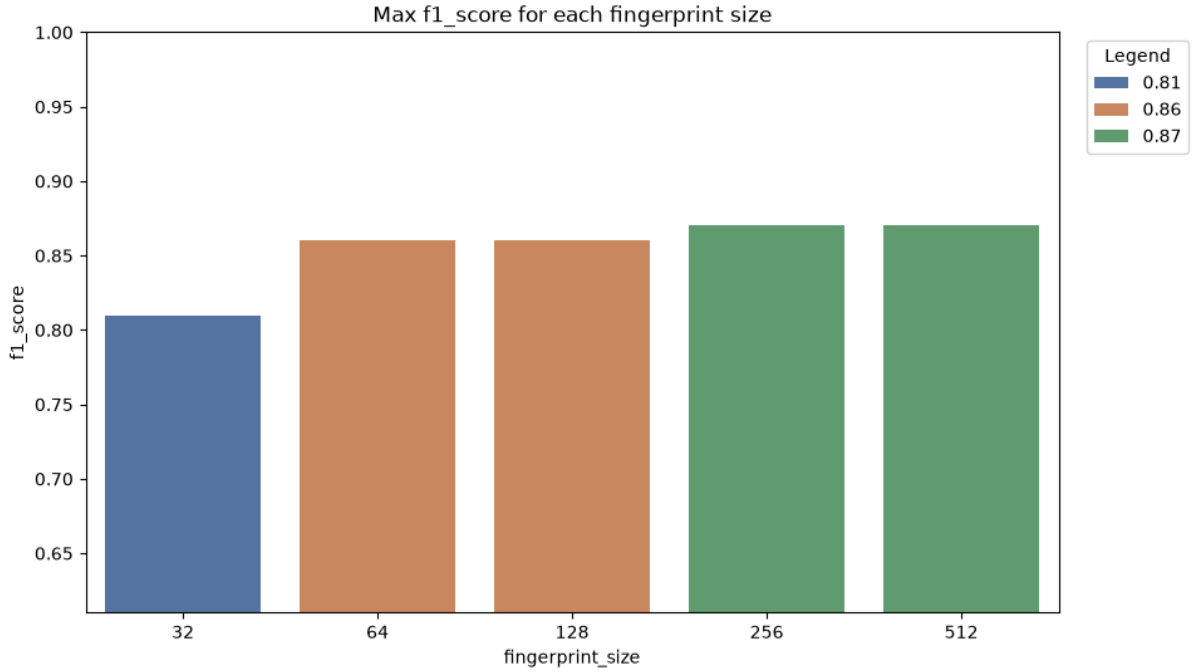
Results

1. Fingerprint Generation

The proposed feature models are compared across two datasets: *llm-paraphrases* and *wikipedia*. Each row of these datasets contains two text columns and a source of truth column, which allows to compare how good is a certain feature model to another. The former dataset is semantic - two texts are considered near-duplicate iff they mean the same thing. The latter is syntactical - rows considered near-duplicate contain the same text, but with around 10% of noise (ie, altered words).

Semantic Duplicate Detection

The semantic duplicate detection benchmark is performed on the llm-paraphrases dataset. In this case the embeddings model reaches the highest score, followed by TF-IDF and bag_of_words. This is within expectations: despite its small size, the embedding model outperforms classic NLP feature model on semantic similarities. Fingerprint sizes of at least 64 bits perform about the same, while 32 bits doesn't seem to be enough to represent features in this case. ...



Syntactical Duplicate Detection

The syntactical duplicate detection benchmark is based on the wikipedia dataset [6], modified for this task. Each row contains two wikipedia entries - the second entry can be a copy of the first one with about 10% of syntax noise, or talk about something completely different. All feature models achieve perfect scores, even on the smallest fingerprint size. ...

Fingerprint Matching Performance

As shown in ..., a dataset of $1,000,000$ 64-bit fingerprints, splitted between corpus and query in a 80:20 ratio, takes about ... seconds to compute all near-duplicates with $K = 3$. This is ran on a MacBook Air M1 laptop. Higher values of K significantly increase the required time, as expected: with the same environment, raising K to 5 takes ... seconds.

Conclusion

The results have shown that the near-duplicate detection algorithm ideated by Manku et al [1] is consistent with literature. Also, it's flexible enough to support advanced feature models like embeddings, detecting semantic near-duplicates for low K values.

Bibliography

[1] A. D. S. Gurmeet Singh Manku Arvind Jain, "Detecting Near-Duplicates for Web Crawling," 2007.

[2] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002.

[3] [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html

[4] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," *arXiv preprint arXiv:2506.05176*, 2025.

[5] O. Ertl, "Speeding up SimHash by 10x using a bit hack," 2023.

[6] W. Foundation, "Wikimedia Downloads." [Online]. Available: https://dumps.wikimedia.org/